

GtelPay Core — Tóm tắt quyết định thiết kế

Một core kế toán + ví (fiat) (`core.accounting` + `core.wallet`). Một trang cho người ra quyết định; tài liệu sâu là phần nền, không đưa vào đây.

Vấn đề

Trong thanh toán, đúng đắn **chính là tiền**. Các kiểu lỗi không phải chuyện hình thức:

- **Tiêu hai lần / double-spend** — member tiêu cùng khoản tiền hai lần (retry, race, timeout).
- **Sổ lệch hoặc sửa được** — không tin được sổ cái để kiểm toán hay với cơ quan quản lý.
- **Mâu thuẫn nhanh vs đúng** — “member tiêu được bao nhiêu *bây giờ?*” cần đọc mili-giây; “đã xảy ra gì trong sổ cái?” cần kế toán bất biến, kiểm toán được. Ép vào một mô hình thì một trong hai vỡ.

Một bảng ví CRUD thông thường không xử lý an toàn cái nào. Cần một core xây cho tiền.

Cách tiếp cận (ba ý tưởng)

1. **Tách hai domain có chủ đích.** `core.wallet` = sổ dư tiêu được nhanh (hot path). `core.accounting` = sổ cái ghi kép bất biến (nguồn sự thật). Nhu cầu nhất quán ngược nhau → tách riêng, không chung storage.
2. **Đúng đắn ngay từ thiết kế.** Mọi thay đổi sổ dư đều **idempotent** và **ghi trong cùng transaction** (không sổ dư nào đổi mà thiếu dòng giao dịch). Sổ cái luôn **cân** (nợ = có) và **không sửa** — sai thì ghi bút toán đảo. Tài khoản trung chuyển về 0 ở mỗi luồng hoàn tất.
3. **Điều phối, không phải distributed transaction.** Một sequencer mỏng điều phối hai domain bằng **saga + bù trừ** — không 2-phase commit mong manh.

“**Vậy có over-engineering không?**” — Không. Đây là phép thử.

Over-engineering = phức tạp mà **không đem lại gì**. Ở đây mỗi mảnh đều ứng với một **kiểu mất tiền cụ thể**. Bỏ nó đi không “đơn giản hơn” — mà thành một rủi ro có tên.

Thành phần	Lỗi nó ngăn	Cái giá nếu bỏ
Idempotency (dưới lock)	Retry / race → tiêu / ghi có hai lần	Mất tiền trực tiếp mỗi đợt retry dồn
Ghi kép + sổ cái bất biến	Sổ lệch hoặc bị sửa	Rót kiểm toán / cơ quan quản lý ; không chứng minh được sổ dư
Tách wallet ⊥ accounting	Một mô hình không thể vừa nhanh vừa bất biến-kiểm-toán	Hoặc tiêu chậm, hoặc sổ cái không tin được
Trung chuyển về 0	Tiền kẹt giữa luồng, không track	Lệch âm thầm, đối soát vỡ
Saga + bù trừ	Lỗi giữa chừng xuyên domain làm tiền mất nhất	Tiền kẹt/nhân đôi khi có sự cố

Câu hỏi trung thực không phải “cái này có phức tạp không?” — mà là “anh muốn cắt lá chắn nào, và chấp nhận mất mát kiểu gì?” Mỗi cái là chuẩn tối thiểu cho hệ thống tiền (cùng bộ primitive mà ngân hàng, Stripe, Modern Treasury đều xài).

Những thứ tụi mình CỐ Ý không xây (bằng chứng tiết chế)

Over-engineering thật sự là mạ vàng. Tụi mình từ chối — ghi rõ trong ADR:

- **4 module, không phải 40** — một shared lib tối thiểu (ADR-002 từ chối hẳn module “common” thứ hai).
- **Không 2PC, không Temporal** — RabbitMQ worker thường (ADR-035); saga qua network seam, không gì nặng hơn.
- **Không tự chế DB sổ cái** — PostgreSQL thường, hai schema (ADR-003).
- **Không event-sourcing, không CQRS** — sổ dư snapshot + log chỉ-thêm, vậy thôi (ADR-004).
- **Một tiền tệ (VND) v1, tính năng opt-in** — phạm vi giữ nhỏ có chủ đích (ADR-019); không xây gì “phòng hờ”.

Một thiết kế over-engineering thật sự sẽ không có danh sách “đã từ chối” dài như vậy.

Vì sao đáng tin — không phải giấy, nó chạy

- `core.foundation`, `core.wallet`, `core.accounting`, `app-orchestration` đã **build, test pass** (Java / Spring Boot 3) — không phải slide.
- Các luật trọng yếu về tiền được **code và test**: sổ dư 1 dòng làm chủ + log chỉ-thêm trong cùng transaction; **idempotency check lại dưới lock** (retry không áp hai lần); journal cân + bất biến; nạp tiền hai pha (chỉ credit sau khi sổ cái post).
- **41 ADR** khóa từng quyết định kèm tiêu chí nghiệm thu + test case; **bộ invariant SQL gác build** (mọi journal cân, trung chuyển về 0) — lịch là rớt CI, không lọt production.

Tự xây hay mua sẵn

Đã so với các ledger có sẵn (Blink, Formance, TigerBeetle). Chúng **gộp sổ dư và sổ cái vào một mô hình** — rất tốt cho ledger nhúng. Nhu cầu của mình (chart of accounts thật, trung chuyển về 0, settlement cuối ngày, audit chuẩn cơ quan quản lý, và một sổ dư tiêu được RPS cao) cần **tách riêng**. Quyết định: **mượn pattern đã được chứng minh, tự xây phần phải khớp thực tế ngân hàng/kiểm toán**. Phần build nhỏ, tập trung, và đã chạy.

Rủi ro được kiểm soát

- **Tương thích ngược** — tính năng mới là opt-in; client cũ không bị ảnh hưởng.
- **Đối soát chỉ báo cáo** — phát hiện lệch, không âm thầm sửa sổ.
- **Hợp đồng API (OpenAPI / AsyncAPI) là nguồn sự thật** — wire không lệch khỏi thiết kế.

Phạm vi

`core.accounting` (sổ cái) + `core.wallet` (sổ dư member) + `core.foundation` (shared lib) + `app-orchestration` (sequencer). **Fiat (VND)**. Toàn bộ ADR, luồng nghiệp vụ, contracts, và 150+ kịch bản nghiệm thu có sẵn làm tài liệu nền, cung cấp khi cần.

Nền: *adr/ (41 ADR + AC/TC) · spec/ (foundation, processes, contracts) · design–v2/acceptance.md (kịch bản nghiệm thu) · platform/ (module chạy + test).*